

Justificación Técnica del Proyecto: Logística del Transporte de Café

1. ¿Por qué el Backend está organizado así? (FastAPI + Arquitectura en Capas)

El Backend es el núcleo del sistema. Se encarga de procesar la información enviada por los conductores, gestionar la trazabilidad del café, almacenar las ubicaciones GPS y administrar los usuarios.

Para mantener el proyecto organizado se utiliza una Arquitectura en Capas, donde cada carpeta tiene una responsabilidad específica:

Models (Modelos)

Representan la estructura de la base de datos.

Aquí se definen entidades como:

- Usuario
- Conductor
- Vehículo
- Solicitud
- Carga
- Viaje
- GPS
- Historial de Eventos

Su función es indicar qué información se almacena y cómo se relaciona.

Schemas (Esquemas)

Validan los datos que ingresan y salen del sistema.

Por ejemplo:

- Verificar que una ubicación GPS tenga latitud y longitud válidas.
- Validar que una solicitud tenga toda la información requerida.

Esto ayuda a evitar errores y mantener la calidad de los datos.

Repositories (Repositorios)

Son los encargados de comunicarse directamente con PostgreSQL.

Si el sistema necesita:

- Guardar una ubicación GPS.
- Consultar un viaje.
- Registrar un evento.

La operación se realiza a través de un repositorio.

Services (Servicios)

Contienen las reglas de negocio.

Por ejemplo:

- Iniciar un viaje.
- Finalizar una carga.
- Validar el estado de una solicitud.
- Procesar la sincronización de datos enviados desde el celular.

Esto permite separar la lógica del negocio de la lógica de acceso a datos.

API / Routers (Enrutadores)

Funcionan como puntos de acceso al sistema.

Reciben las peticiones de:

- La aplicación móvil.
- El sistema web.

Y las envían al servicio correspondiente.

¿Por qué FastAPI?

Se eligió FastAPI porque:

- Tiene un alto rendimiento.
- Maneja múltiples solicitudes simultáneamente.
- Facilita la creación de APIs REST.
- Se integra fácilmente con PostgreSQL.
- Permite implementar WebSockets para monitoreo en tiempo real.

Esto es ideal para un sistema que recibe constantemente datos GPS desde vehículos en movimiento.

2. ¿Por qué el Frontend está organizado así? (React Native)

La aplicación móvil será utilizada por los conductores durante el transporte del café.

Se eligió React Native porque permite desarrollar una única aplicación para Android e iOS utilizando una sola base de código.

La estructura se divide en módulos para facilitar el mantenimiento.

Screens (Pantallas)

Representan las vistas principales de la aplicación.

Por ejemplo:

- Login
- Dashboard
- Viaje
- Eventos
- Historial
- Sincronización

- Perfil

Cada pantalla tiene una función específica dentro del proceso logístico.

Components (Componentes)

Contienen elementos reutilizables.

Por ejemplo:

- Tarjetas de viaje.
- Indicadores de conexión.
- Estado del GPS.
- Formularios de eventos.

Esto evita duplicar código y mejora la consistencia visual.

Services (Servicios)

Gestionan la comunicación con el Backend.

Permiten:

- Iniciar sesión.
 - Enviar eventos.
 - Consultar viajes.
 - Sincronizar información.
-

Contexts y Hooks

Permiten compartir información entre distintas pantallas.

Por ejemplo:

- Usuario autenticado.
- Estado del GPS.

- Estado de sincronización.

Esto mejora la experiencia de uso y simplifica el desarrollo.

3. La Estrategia Offline: ¿Qué pasa si el vehículo pierde conexión?

El transporte de café suele realizarse en zonas rurales donde la cobertura móvil puede ser limitada.

Por esta razón se implementa una estrategia Offline First, que permite continuar operando sin conexión a internet.

1. Almacenamiento Local (SQLite)

Si el conductor:

- Registra un evento.
- Inicia un viaje.
- Captura una ubicación GPS.

Y no existe conexión a internet, los datos se almacenan automáticamente en SQLite dentro del dispositivo móvil.

De esta manera no se pierde información.

2. Cola de Pendientes

Los registros almacenados se marcan como pendientes de sincronización.

Estos datos permanecen guardados localmente hasta que exista conectividad nuevamente.

3. Sincronización Automática

El sistema monitorea continuamente el estado de la red.

Cuando detecta conexión:

- Envía las ubicaciones GPS pendientes.

- Envía los eventos registrados.
- Actualiza la información del servidor.

Todo este proceso ocurre automáticamente sin intervención del conductor.

4. Seguimiento GPS y Trazabilidad en Tiempo Real

Uno de los objetivos principales del proyecto es conocer la ubicación de los vehículos y garantizar la trazabilidad del transporte del café.

GPS

La aplicación captura periódicamente:

- Latitud.
- Longitud.
- Fecha y hora.
- Estado del viaje.

Esta información permite reconstruir todo el recorrido realizado.

WebSockets

Se utilizan WebSockets para transmitir información en tiempo real.

A diferencia de las consultas tradicionales donde el sistema pregunta constantemente por nuevas ubicaciones, los WebSockets permiten que el servidor envíe actualizaciones automáticamente.

Esto mejora la velocidad y reduce el consumo de recursos.

PostgreSQL + PostGIS

Se eligió PostgreSQL como base de datos principal debido a su estabilidad y rendimiento.

Además, se incorpora PostGIS para trabajar con información geográfica.

Gracias a PostGIS el sistema puede:

- Almacenar coordenadas GPS.
- Calcular distancias recorridas.
- Analizar rutas.
- Implementar geocercas.
- Generar reportes geográficos.

Estas capacidades son fundamentales para un sistema de logística y trazabilidad de transporte de café.

Conclusión

La arquitectura seleccionada permite construir un sistema escalable, organizado y confiable para la gestión logística del transporte de café. La combinación de React Native, SQLite, FastAPI, PostgreSQL y PostGIS garantiza que la aplicación continúe funcionando incluso en zonas sin cobertura, manteniendo la trazabilidad de la carga y proporcionando monitoreo en tiempo real de los vehículos.